

Highlights

What Drives Small Object Detection Accuracy in UAV Imagery? A Controlled Study of Training Resolution, Detection Stride, and Feature Modules

Moatassim Barakat, Xinzhong Zhu

- Single-resolution training beat a four-phase progressive recipe by 3–4 pp
- Its gain came mostly from medium and large objects, not from small ones
- A stride-4 level gained 3–7 AP below 16 px and lost 1–2 AP above 32 px
- Edge selection and recalibration did not improve small objects on two seeds
- Neither multi-scale dilation nor adaptive gating beat a plain convolution

What Drives Small Object Detection Accuracy in UAV Imagery?

A Controlled Study of Training Resolution, Detection Stride, and Feature Modules

Moatassim Barakat^a, Xinzhong Zhu^{a,*}

^a*College of Computer Science and Technology, Zhejiang Normal University, 688 Yingbin Road, Jinhua, 321004, China*

Abstract

Small-object detectors for drone imagery are usually built by attaching modules to a YOLO baseline, with gains of one to three points from single runs. We asked which design decisions matter once budget, evaluation settings and seed are controlled. Using YOLOv11s on VisDrone-DET2019, we crossed progressive and single-resolution training at a matched epoch budget with heads with and without a stride-4 level, and tested multi-scale edge information selection (MEIS) with CBAM-style recalibration. Differences were tested with paired bootstrap tests, resolved by object size. Single-phase training at 1280px beat the progressive schedule by 3.3 points mAP@50 without the stride-4 level and 3.9 with it ($p < 0.002$; 4.2 on a second seed), with the gain mainly on medium and large objects; the schedules also differ in learning-rate restarts and augmentation, so this compares recipes, not resolution scheduling alone. The stride-4 level raised AP@50 by 3 to 7 points below 16 input pixels and lowered it by 1 to 2 points above 32, so mAP@50 rose only 0.7 points. MEIS changed mAP@50 by +0.6 and -0.3 points on two seeds and improved no size range below 32 pixels; a factorial ablation found neither multi-scale dilation nor adaptive gating better than a plain convolution. A sampling account fits the architectural results but not the schedule effect: modules after downsampling cannot restore what the stride discards. On UAVDT without retraining the models kept their car order-

*Corresponding author.

Email address: zxz@zjnu.edu.cn (Xinzhong Zhu)

ing, though eleven sequences could not resolve it; UAVDT has only vehicle classes, so small non-vehicle generalization remains untested.

Keywords: Small object detection, UAV imagery, Sampling resolution, Training schedule, Ablation study, Statistical significance, YOLOv11

1. Introduction

Drones photograph scenes from tens to hundreds of metres above the ground, so the objects that matter for traffic monitoring, crowd counting or search and rescue often cover only a few dozen pixels [1, 2]. In the VisDrone-DET2019 training set, 63% of the annotated instances have a geometric-mean side shorter than 32 pixels in the original frame. After the frame is resized for the network, such an object falls on a handful of cells of a stride-8 feature map, and a detector that sees only a few samples of an object has little to work with, however its later layers are designed.

Much of the recent work on this problem modifies the network. Attention blocks, edge enhancement branches, weighted feature fusion and extra prediction heads have all been attached to YOLO baselines [3, 4, 5], and the reported improvements are usually modest; MEIS-YOLO, for example, reports 1.4 points of AP@50 over its YOLOv11 baseline [4]. Two things make gains of that size hard to interpret. If the proposed model and its baseline are not trained under the same conditions, a gain that comes from a longer schedule or a larger input can end up credited to the new module. And networks trained from different random seeds reach different accuracies [6]; for detectors on VisDrone we measured seed-to-seed differences of up to 0.7 points, which is comparable to the effects being claimed.

This paper reports what happened when we removed those confounds. We started from YOLOv11s [7] and three modules that target the usual explanations for poor small-object accuracy: a multi-scale edge information selection (MEIS) block built from dilated depthwise-separable branches with input-adaptive gating, following the idea of MEIS-YOLO [4]; a cross-scale weighted fusion neck in the spirit of CF-YOLO [5] and BiFPN [8]; and a CBAM-style recalibration module [9] placed after fusion. We also studied two changes that act on sampling rather than on features: the input resolution used for training, reached either directly or through a four-phase progressive schedule, and a stride-4 (P2) detection level. Every comparison uses a matched epoch budget and one evaluation protocol. The module

comparison and both schedules of the four-level detector were repeated with independent seeds, and all differences were tested with paired resampling over the validation images and broken down by object size.

The results did not follow the module-centred story we expected. Single-resolution training at 1280 px was 3.3 to 4.2 points better than the progressive schedule at the same epoch budget, with and without a stride-4 level, and it reached the schedule’s final accuracy with one third of its compute. Split by object size, however, this gain came mostly from medium and large objects. The stride-4 level behaved as a change in sampling should: it raised accuracy on objects spanning fewer than two stride-8 cells and lowered it slightly on large objects, which left overall mAP almost unchanged. MEIS with recalibration produced no significant change on either of two seeds and no gain for small objects, and a factorial ablation showed that neither of the two ingredients that define MEIS, multi-scale dilation and input-adaptive gating, did better than a plain convolution with the same number of parameters. Section 3 sets out the sampling argument behind these tests. It accounts for the stride and module results but not for the schedule effect.

The paper makes three contributions: (i) a matched-budget comparison of progressive and single-resolution training for small-object detection, crossed with the detection stride, with measured compute and paired significance tests; (ii) a controlled evaluation of edge selection, weighted fusion and recalibration modules, including a factorial ablation that separates multi-scale dilation, adaptive gating and added capacity; and (iii) a sampling account that gives, for a given input size and stride, the smallest object that can receive positive training samples, tested with a size-resolved evaluation that supports it for the detection stride and the modules and rules it out as the explanation of the schedule effect.

2. Related Work

2.1. Feature modules for small aerial objects

Feature pyramids [10, 11] and weighted bidirectional fusion [8] are the standard way to give a detector features at several scales, and channel or spatial attention [12, 9, 13] is often added to suppress the cluttered backgrounds of aerial scenes. Detectors designed for drone imagery combine these ideas. TPH-YOLOv5 [3] adds transformer prediction heads, CBAM and an extra prediction head for tiny objects. MEIS-YOLO [4] inserts an edge information selection block, an asymptotic feature pyramid and bi-level routing

attention into YOLOv11. CF-YOLO [5] replaces the YOLOv5s neck with cross-fusion blocks for detection in snow. Detection transformers have been adapted to the same setting: UAV-DETR [14] adds frequency-domain feature fusion, a detail-preserving downsampling module and a semantic alignment step to RT-DETR [15], and reports +4.2 points AP@50 on VisDrone over its baseline. These papers report their gains without estimates of run-to-run variation, which makes differences of a point or two difficult to judge. When a design adds a higher-resolution head together with new blocks, as TPH-YOLOv5 does, the change in sampling alone may account for part of the gain. The improvements they claim, two to four points, are of the same order as the effect we measure for the training schedule alone (Section 5.1), which is why this paper varies one factor at a time rather than adding another architecture to the comparison.

2.2. Resolution and sampling in aerial detection

A second line of work treats small objects as a resolution problem. Coarse-to-fine detectors such as ClusDet [16], DMNet [17] and UFPMP-Det [18] locate object-dense regions and process them at higher magnification. Query-Det [19] runs a cheap coarse pass and computes high-resolution features only where small objects are likely, and SAHI [20] slices large images into overlapping tiles. These methods show that magnification helps, but they also change the inference pipeline and its cost. We look at the two simplest sampling controls available inside a single-pass detector: the input resolution used in training and the stride of the finest detection level.

2.3. Resolution schedules during training

Curriculum learning [21] orders training from easy to hard, and progressive resizing applies a similar idea to image resolution. EfficientNetV2 [22] trains image classifiers on small images first and raises resolution and regularization together, mainly to shorten training. Touvron et al. [23] showed that test-time resolution interacts with training resolution, so a classifier can benefit from being tested at a larger size than it was trained on. Both lines of work concern classification. For dense detection of tiny objects, where a low training resolution can shrink objects below the size a detection level can represent, we are not aware of a matched-budget comparison between progressive and single-resolution training.

2.4. Variance and significance in detector evaluation

Networks trained with different seeds, data orders and initializations reach different accuracies, and Bouthillier et al. [6] showed that ignoring this variation leads to unreliable conclusions in machine learning benchmarks. When a metric is computed over a test set, paired resampling tests [24, 25] offer a practical check, because both models are scored on the same resampled images. Detection papers rarely report them. We apply them at the image level and report separately how much results move between seeds.

3. A Sampling View of Small Object Detection

A detector does not see an object at its original size. A frame whose longer side is W_0 pixels is resized so that this side equals the network input size S , which scales an object of side a to ka with $k = S/W_0$. A feature level with stride r then samples the resized image once every r pixels, so the number of samples across the object is

$$n = \frac{ka}{r} = \frac{Sa}{rW_0}. \quad (1)$$

Seen as a sampling problem, n is the sampling rate relative to the object. With fewer than about two samples per side, a level cannot represent the object’s spatial structure, and the layers that follow inherit the loss [26, 27].

For anchor-free YOLO detectors the consequence is more direct. YOLOv11 assigns training targets with the task-aligned assigner of TOOD [28], which only considers anchor points whose centres lie inside a ground-truth box. At a given level an object covers roughly $n_w n_h$ anchor points, where n_w and n_h count samples along its width and height. When both counts are below one, the object may contain no anchor point at that level. If that happens at every level, the object contributes no positive sample and is learned only as background. Setting n to one or two in Eq. (1) gives the smallest object side that can be assigned a target, and the smallest that is sampled twice per side:

$$a_{\min} = \frac{n r W_0}{S}. \quad (2)$$

Fig. 1 illustrates the effect and Table 1 evaluates Eq. (2) for frames 2000 pixels wide, which is about the size of the largest VisDrone-DET2019 frames. Halving the stride and doubling the input size have the same effect on $a_{\min}$. Measured directly in network-input pixels, the geometric-mean side $\sqrt{wh}$ of

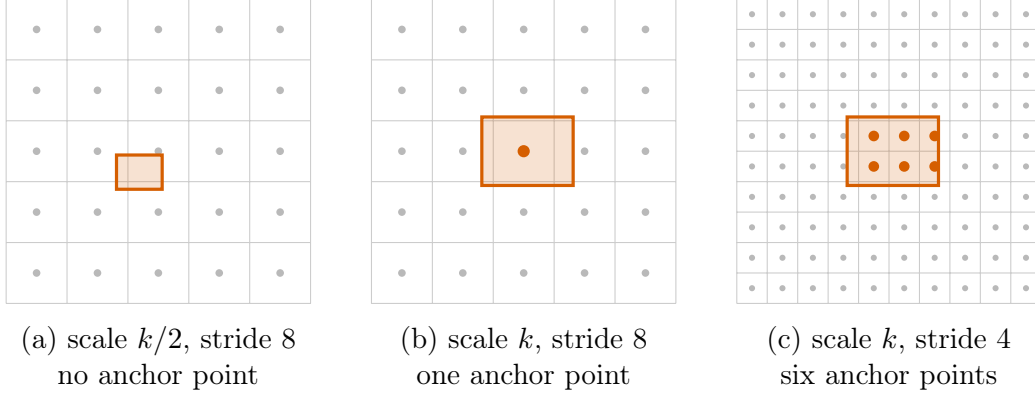


Figure 1: The same object (red box) under three sampling settings. Grid cells span one stride in network-input pixels and dots mark anchor points; filled red dots lie inside the object and can be assigned as positive samples. Halving the input scale (a) removes the only anchor point that the object covers at stride 8 (b), while a stride-4 level (c) places six anchor points on it.

Table 1: Smallest object side $a_{\min}$, in pixels of a 2000-pixel-wide frame, that yields at least n anchor points per side (Eq. 2).

Stride r	n	Network input size S (px)				
		640	800	1024	1280	1536
4 (P2)	1	12.5	10.0	7.8	6.3	5.2
	2	25.0	20.0	15.6	12.5	10.4
8 (P3)	1	25.0	20.0	15.6	12.5	10.4
	2	50.0	40.0	31.3	25.0	20.8

31.0% of the VisDrone validation instances is smaller than one stride-8 cell at an input size of 640 px, and that of 5.8% is smaller than one stride-4 cell. At 1280 px, 5.8% of the instances (the same objects) are smaller than one stride-8 cell and 0.2% smaller than one stride-4 cell.

The argument leads to three predictions that the experiments can check.

- P1. Training at a larger input size, or adding a finer detection level, should help most on small objects, because both raise n for the same frames. A stride-4 level in particular should help objects that span fewer than about two stride-8 cells and matter little for larger ones.
- P2. A module that operates on stride- r features cannot raise n . It can

reweight what was sampled, but it cannot give training signal to objects that received no positive samples, so it should not improve small objects the way a finer stride does.

- P3. A progressive schedule that starts at low resolution spends its early epochs in a regime where many small objects cannot be assigned targets (Table 1, 640 and 800 px). Under a fixed epoch budget it should therefore do worse than training at the final resolution throughout, and the deficit should be largest for small objects.

Section 5.5 tests all three predictions by object size. Eq. (2) is a simplification. Mosaic and scale augmentation change k from one training image to the next, context around an object can support detection, and the assigner’s behaviour near box borders depends on implementation details. We use the argument to organise the experiments, not as a quantitative model of accuracy.

4. Experimental Setup

4.1. Dataset

We use VisDrone-DET2019 [1], which provides 6,471 training images and 548 validation images annotated with ten object categories. All results in this paper are on the validation set; we did not use the test-dev split. Annotations were converted to the YOLO format, keeping categories 1 to 10 and discarding ignored regions (category 0) and the *others* class (category 11). Boxes were clipped to the image, and boxes one pixel wide or high or smaller after clipping were removed. The converted training set contains 305,974 instances and the validation set 38,759. The class distribution is strongly skewed (Fig. 2): cars and pedestrians account for 65.5% of training instances, while tricycles, awning-tricycles and buses together make up 4.3%.

Cross-dataset evaluation. For the zero-shot test of Section 5.6 we use UAVDT [29], a second aerial benchmark recorded at 1080×540 pixels and annotated with three vehicle classes: car, truck and bus. Our copy of its detection benchmark holds 11 of the 50 sequences, four from the official test split and seven from the training split. Since nothing here is ever trained on UAVDT, all eleven can be used, and they were not selected by any result. We keep every fifth frame, because the videos run at 30 frames per second and neighbouring frames are nearly identical, which leaves 1,535 images with 12,450 annotated

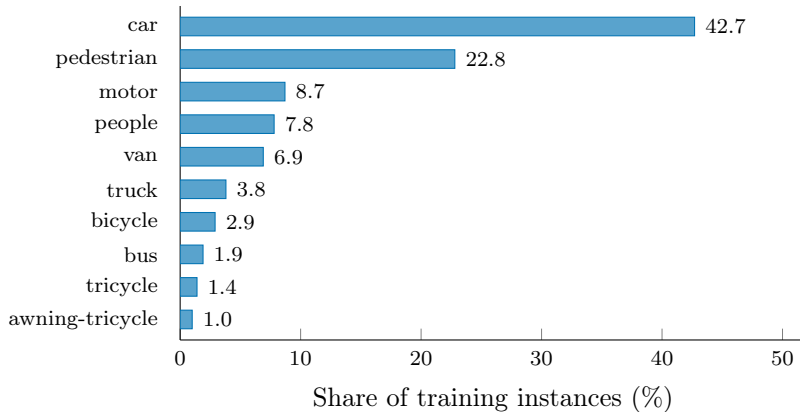


Figure 2: Class distribution of the 305,974 training instances in VisDrone-DET2019 after conversion.

vehicles (11,681 cars, 583 trucks, 186 buses). UAVDT marks regions of vehicles too small to annotate and excludes them from evaluation: we drop ground truth inside those regions and, during evaluation, every detection that lies mostly inside one, about a tenth of all detections. UAVDT has no van class, so "van" detections are relabelled as car and non-maximum suppression is re-applied to the merged class. Its objects are larger than VisDrone's: the median geometric-mean side is 38.5 pixels and 30.7% of the instances fall below 32 pixels, against 68.6% on the VisDrone validation set.

4.2. Detectors and modules

All detectors are built on YOLOv11s [7] and initialized from its COCO-pretrained weights. The stock model, which we call YOLOv11s (P3-P5), detects at strides 8, 16 and 32 and serves as the three-level reference in the stride comparison. The *Baseline-P2* variant extends the top-down path of the neck to the stride-4 backbone features and starts the bottom-up path from there, which gives four detection levels (Fig. 3). Where a modified neck contains C3k2 blocks whose weight shapes match the original head, those weights are copied as well, so that no variant starts from a randomly initialized block when a pretrained counterpart exists.

Multi-scale edge information selection (MEIS). MEIS is applied to each backbone output that feeds the neck. For an input $X \in \mathbb{R}^{C \times H \times W}$, three depthwise-separable branches with dilation $d \in \{1, 2, 4\}$ produce edge responses E_d , and

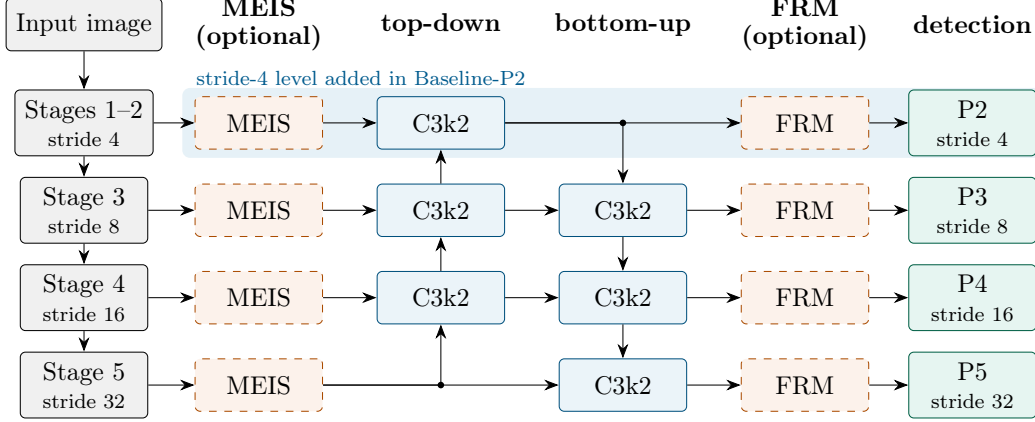


Figure 3: Detector variants. The Baseline-P2 neck extends the YOLOv11s path aggregation neck to the stride-4 backbone features (shaded row). Each C3k2 node in the top-down path follows an upsampling and concatenation, and each node in the bottom-up path follows a strided convolution and concatenation. Dashed blocks are the optional modules evaluated in Section 5.3; Stage 5 includes the SPPF and C2PSA blocks of YOLOv11.

a channel gate computed from each response weights it:

$$g_d = \sigma(W_2 \delta(W_1 \text{GAP}(E_d))) \in [0, 1]^C, \quad Y = X + \sum_d g_d \odot E_d, \quad (3)$$

where GAP is global average pooling, W_1 and W_2 are 1×1 convolutions with reduction ratio 16, δ is ReLU and σ the sigmoid. The gate is recomputed for every input. Our block tests the dilation-and-gating interpretation of edge selection; the MEIS module of MEIS-YOLO [4] instead uses adaptive pooling, high-pass edge enhancement and a dual-domain selection mechanism, so our results do not transfer to that module directly.

Feature recalibration module (FRM). FRM applies the channel and spatial attention of CBAM [9] to each neck output and blends the result residually, $Y = X + \alpha(X_c - X) + \alpha(X_s - X_c)$, where X_c and X_s are the outputs of the channel and spatial attention steps and $\alpha = 0.5$.

Factorial variants. To separate the ingredients of MEIS, we replaced the dilations (1, 2, 4) with (1, 1, 1), which leaves the parameter count unchanged, and replaced the input-adaptive gate with an input-independent learnable channel weight $\sigma(\theta_d)$, $\theta_d \in \mathbb{R}^C$. The weights θ_d are initialized to zero, so both gate types start close to 0.5. The variant with dilations (1, 1, 1) and a

Table 2: Parameters, computational cost and speed of the evaluated detectors.

Model	Levels	Params (M)	GFLOPs at input size			Images/s, batch 1		Images/s, batch 8	
			640	1280	1536	1280	1536	1280	1536
YOLOv11s (P3–P5)	3	9.43	21.6	86.3	124.2	89.3 [†]	88.1 [†]	183.6	125.5
Baseline-P2	4	9.92	37.5	149.8	215.7	75.7 [†]	63.5 [†]	97.1	67.1
Baseline-P2 + MEIS + FRM	4	11.40	44.4	177.4	255.5	40.3	28.6	52.5	36.9
dilation (1,1,1), adaptive gate	4	11.40	44.4	177.4	255.5	–	–	–	–
dilation (1,2,4), static gate	4	11.25	44.3	177.3	255.3	–	–	–	–
dilation (1,1,1), static gate	4	11.25	44.3	177.3	255.3	–	–	–	–

GFLOPs are twice the multiply–accumulate operations for one square image with unfused convolution and batch-normalization layers. Throughput: one NVIDIA RTX 3080 with no other process on the machine, FP16, fused convolution and batch-normalization layers, network forward pass only, median of five timed runs. [†]With one image per batch these two models are limited by how fast the host can submit work, not by the GPU: their forward pass takes 11.2 and 13.2 ms whatever the input size. The batch-8 columns, where the time does grow with the pixel count, are the comparison between architectures.

static gate is a plain residual block of depthwise-separable convolutions with the same capacity as MEIS and no multi-scale or adaptive component.

Cross-scale weighted fusion. This neck, evaluated only in the earlier experiments of Appendix [Appendix A](#), combines resized and channel-projected pyramid features $\tilde{F}_i$ with normalized learnable weights, $F = \sum_i \frac{\text{ReLU}(w_i)}{\sum_j \text{ReLU}(w_j) + \epsilon} \tilde{F}_i$, followed by a depthwise-separable convolution, similar to the fast normalized fusion of BiFPN [8].

Table 2 lists the size and computational cost of each variant. Measured throughput and operation counts do not agree. The stride-4 level costs 74% more operations and 89% more time, but MEIS with FRM costs 18% more operations and 85% more time, so its measured cost is 1.6 times what its operation count implies. Its dilated branches and gating act on the stride-4 map, where the arithmetic per loaded value is low, and they add many small kernels; neither shows up in a FLOP count. The two effects compound: the full model runs 3.5 times slower than YOLOv11s at 1280 px for 2.1 times the operations.

4.3. Training

Table 3 lists the two schedules. The *progressive* schedule trains for 100, 100, 100 and 50 epochs at 640, 800, 1024 and 1280 px. Each phase restarts a cosine learning-rate schedule with warm-up, and augmentation strength decreases while the box and classification loss weights increase from phase to

Table 3: Training schedules. Learning rates follow a cosine curve from lr_0 to $\text{lr}_0 \cdot f$ within each phase.

	Input (px)	Epochs	Batch (P2 / P3)	lr_0	f	Warm-up (ep.)	Loss weights box / cls / DFL	Mosaic / mixup / copy-paste	Scale / rotation
Progressive, phase 1	640	100	8 / 16	0.01	0.01	5	7.5 / 0.5 / 1.5	1.0 / 0.15 / 0.3	$0.9 / 10^\circ$
Progressive, phase 2	800	100	6 / 12	0.005	0.05	3	8.0 / 0.6 / 1.6	0.8 / 0.10 / 0.4	$0.8 / 7^\circ$
Progressive, phase 3	1024	100	4 / 8	0.002	0.08	2	8.5 / 0.7 / 1.7	0.5 / 0.05 / 0.2	$0.7 / 5^\circ$
Progressive, phase 4	1280	50	2 / 6	0.0005	0.10	1	9.0 / 0.8 / 1.8	0 / 0 / 0.1	$0.6 / 2^\circ$
Single phase	1280	350	2 / 4	0.01	0.01	5	7.5 / 0.5 / 1.5	1.0 / 0.15 / 0.3	$0.9 / 10^\circ$

Batch sizes are given for the four-level detectors (P2) and for YOLOv11s with three levels (P3). Mosaic augmentation is switched off for the last 20, 15, 10 and 0 epochs of phases 1–4, and for the last 20 epochs of the single-phase schedule.

phase. The *single-phase* schedule trains for up to 350 epochs at 1280px with the phase-1 settings. Both use SGD with momentum 0.937 and weight decay 5×10^{-4} , mixed precision, and the exponential moving average of weights provided by the Ultralytics trainer. Physical batch sizes were set by GPU memory and differ between the three- and four-level detectors (Table 3). The trainer accumulates gradients to a nominal batch of 64 images, which gave an effective batch of 60 to 66 images in every phase of every run, so the physical batch size mainly changed the number of images over which batch-normalization statistics were computed. Early stopping with a patience of 200 epochs affected only the single-phase YOLOv11s run, which stopped at epoch 345, 200 epochs after its best checkpoint.

We report training compute in pixel-epochs, $C = \sum_e S_e^2$, the sum over epochs of the squared input size. Convolutional cost grows with the number of input pixels, so C compares schedules independently of hardware and of other jobs sharing the machine. The progressive schedule costs 2.92×10^8 pixel-epochs and 350 epochs at 1280px cost 5.73×10^8 .

Under the progressive schedule, Baseline-P2 and Baseline-P2 with MEIS and FRM were trained with seeds 0 and 42, and YOLOv11s with seed 0. Under the single-phase schedule, YOLOv11s was trained with seed 0 and Baseline-P2 with seeds 0 and 42. The factorial ablation of Section 5.3.1 used a shortened progressive schedule with 25, 20, 20 and 10 epochs per phase, seeds 0 and 42, and was otherwise identical. Experiments ran on NVIDIA RTX 3080 GPUs (20 GB) with Ultralytics 8.4, PyTorch 2.5.1 and CUDA 12.1.

4.4. Evaluation

All models were evaluated with the Ultralytics validator using letter-box resizing to input size S , a confidence threshold of 0.001, non-maximum

suppression at an IoU of 0.5 and at most 600 detections per image. The validation images contain 70.7 annotated instances on average. For every run we evaluated the checkpoint with the best validation fitness recorded during training, as the Ultralytics trainer selects it. We report mAP@50 and mAP@50:95, and precision and recall at the confidence threshold that maximizes the mean F1 score. AP follows Ultralytics 8.4: the precision envelope is sampled at 101 recall levels, set to zero beyond the highest recall reached, and integrated with the trapezoidal rule. Some earlier releases, including Ultralytics 8.3.91, extend the curve to a recall of one, which for identical detections raised mAP@50 by 0.5 to 0.7 points on our models. The primary input size is 1280 px, the final training resolution of both schedules; we also report 1536 px.

Accuracy by object size. To test the sampling account, we split AP@50 by object size. For each validation image we recorded the detections and, from inside the validator, the ground-truth box each detection was matched to, so the matching is the validator’s own and the all-sizes result reproduces the reported mAP@50. Object size is the geometric-mean side $\sqrt{wh}$ of a box. We use it in original-image pixels for the COCO ranges [30] (small below 32, medium 32 to 96, large above 96) and in network-input pixels at the evaluated size for four ranges, 4–8, 8–16, 16–32 and at least 32 pixels, which relate directly to stride-4 and stride-8 cells. Within a range we follow the COCO convention: ground truth outside the range is ignored, as are detections matched to it and unmatched detections whose own size lies outside the range. Several classes have only a few boxes in some ranges, so we average AP over the classes with at least 100 ground-truth boxes in the range. This keeps all ten classes for small and medium objects, cars and vans for large objects, and between four and ten classes for the input-pixel ranges.

4.5. Statistical testing

Differences between two models were tested on the 548 validation images with two paired procedures. Per-image detection statistics were captured from the validator with batch size one, and recomputing mAP from them reproduces the validator’s reported value. In the *paired bootstrap* [24], the images were resampled with replacement 1000 times, both models were scored on each resample, and the 2.5th and 97.5th percentiles of the difference form the 95% confidence interval. The two-sided p -value is twice the smaller fraction of resampled differences lying on either side of zero; when

no resample crosses zero we report $p < 0.002$, the smallest value that 1000 resamples can resolve. Contrasts that involve more than two models, such as the interaction between schedule and stride, and all size-resolved differences use the same image resamples for every model. In the *paired permutation test*, the two models’ statistics were swapped independently for each image with probability 0.5, 1000 times, and $p = (m + 1)/1001$, where m counts permutations whose absolute difference in mAP@50 is at least the observed one. These tests describe uncertainty from the choice of evaluation images for fixed trained models. Variation between training seeds is reported separately, since two seeds are too few for a seed-level interval. On UAVDT the resampling unit is the video sequence rather than the image, because frames within a sequence are nearly identical and resampling them would understate the uncertainty. Because ten classes are tested at once in the per-class comparisons, we treat a class difference as significant only when it survives a Bonferroni correction ($p < 0.005$).

5. Results

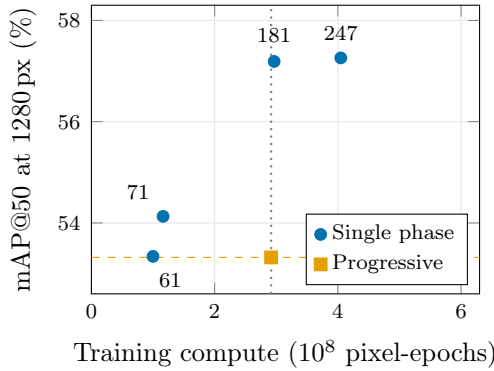
5.1. Progressive versus single-phase training

Table 4 compares the two schedules on Baseline-P2. With seed 0, single-phase training was better on every metric at both evaluation sizes: by 3.9 points mAP@50 and 3.2 points mAP@50:95 at 1280px, and by 4.8 and 3.8 points at 1536px. The paired bootstrap at 1536px gives a 95% confidence interval of [4.1, 5.5] points ($p < 0.002$), and the permutation test gives $p = 0.001$, the smallest value attainable with 1000 permutations. Seed 42 reproduces the result: +4.16 points mAP@50 at 1280px and +5.02 at 1536px. The gap between schedules is an order of magnitude larger than the gap between seeds, which is 0.28 points for the progressive pair and 0.50 for the single-phase pair at 1280px, and every one of the ten classes improves under the single-phase schedule in both seeds and at both evaluation sizes. The effect does not depend on the stride-4 level: YOLOv11s with its stock three-level head gained 3.3 points mAP@50 at 1280px and 4.0 points at 1536px from single-phase training (Table 5; 95% CI at 1536px [3.3, 4.6], $p < 0.002$, permutation $p = 0.001$).

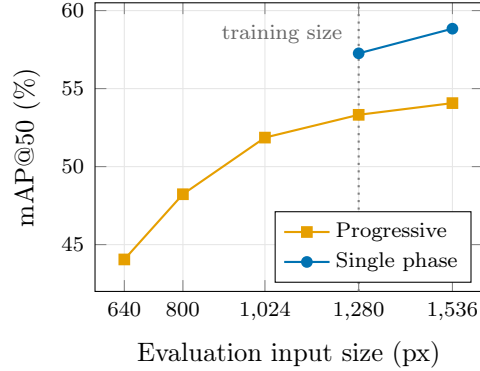
The single-phase run also needed less compute to reach the progressive result. Its checkpoint after 61 epochs, which corresponds to 34.3% of the progressive schedule’s compute, scored 53.34% mAP@50 and 31.66%

Table 4: Progressive and single-phase training of Baseline-P2 on the VisDrone validation set. P and R are precision and recall. Compute is in 10^8 pixel-epochs.

Schedule	Seed	Evaluated at 1280 px				At 1536 px		Compute
		mAP@50	mAP@50:95	P	R	mAP@50	mAP@50:95	
Progressive	0	53.32	31.47	58.9	53.1	54.07	32.01	2.92
	42	53.60	31.51	59.4	52.6	54.26	31.91	2.92
	mean	53.46	31.49	59.2	52.9	54.17	31.96	
Single phase	0	57.26	34.66	62.1	55.1	58.84	35.82	5.73
	42	57.76	34.80	62.7	56.2	59.28	35.88	5.73
	mean	57.51	34.73	62.4	55.7	59.06	35.85	



(a) Accuracy against training compute



(b) Accuracy against evaluation input size

Figure 4: (a) mAP@50 of single-phase checkpoints after 61, 71 and 181 epochs and of the best checkpoint (epoch 247), compared with the finished progressive model (dashed line). The dotted line marks the progressive schedule’s compute. (b) The same seed-0 models evaluated at different input sizes.

mAP@50:95 at 1280 px, against 53.32% and 31.47% for the finished progressive model (Fig. 4a). After 181 epochs, at about the same compute as the whole progressive schedule (101.6%), it reached 57.19%, 3.87 points ahead. That checkpoint comes from the middle of a 350-epoch run whose learning rate had not yet annealed, so a run planned to stop there would probably score at least as high. Later epochs added little: the best checkpoint of the full run, at epoch 247, scored 57.26%, only 0.07 points more.

All ten classes improved (Fig. 5). The largest gain was for bicycles (+7.4 AP@50 at 1280 px), followed by vans and tricycles (+4.7 each); the smallest were for cars (+0.8), which were already above 89 AP, and awning-tricycles

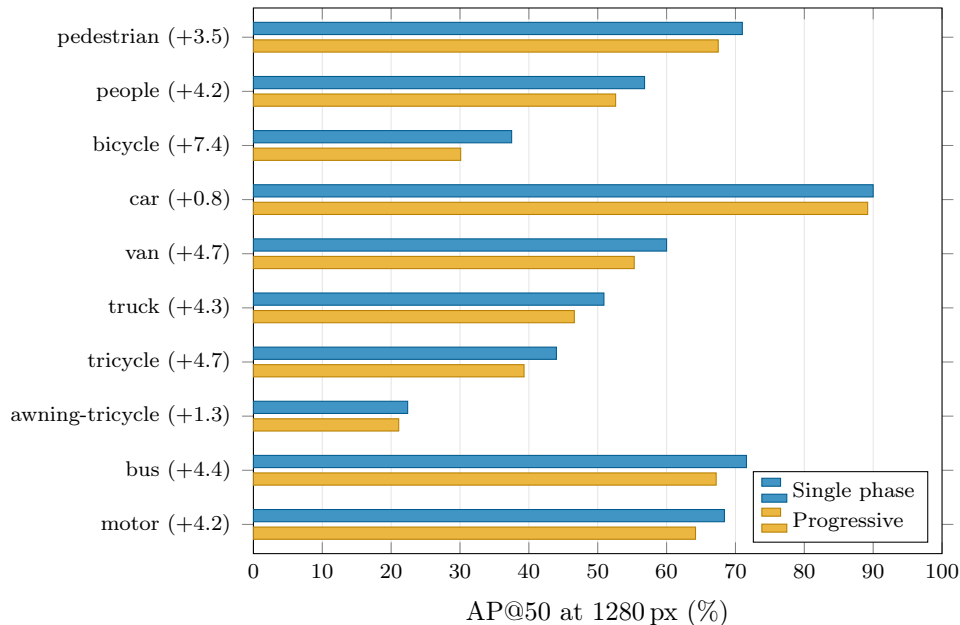


Figure 5: Per-class AP@50 of Baseline-P2 at 1280 px (seed 0). Numbers in parentheses give the gain from single-phase training.

(+1.3). Bicycles are among the thinnest objects in the dataset, but large vehicles such as buses and trucks also gained more than four points, so per-class results cannot show where the gain comes from. Measured by object size (Section 5.5), it was concentrated on medium and large objects.

Both models kept improving as the evaluation size grew (Fig. 4b). The progressive model rose from 44.1% at 640 px to 54.1% at 1536 px, including a gain of 0.8 points from 1280 to 1536 px, above its final training size. The single-phase model gained 1.6 points over the same step. This is consistent with the sampling argument and with the train-test resolution effect described by Touvron et al. [23], although neither explanation is tested directly by these two points.

5.2. Detection stride

Table 5 crosses the two schedules with the two detection heads, all trained with seed 0. Under single-phase training the stride-4 level added 0.73 points mAP@50 at 1280 px (95% CI [0.14, 1.26], $p = 0.012$; permutation $p = 0.019$) and 0.46 points mAP@50:95 ([0.05, 0.83], $p = 0.028$). At 1536 px the difference shrank to 0.37 points ([−0.14, 0.88], $p = 0.16$), and under the progres-

Table 5: Training schedule crossed with the detection head (seed 0). The lower block gives differences between the rows above. Both P2–P5 cells were repeated with seed 42 (Table 4), where the schedule difference is +4.16 points mAP@50 at 1280 px and +5.02 at 1536 px; the P3–P5 cells rest on one seed.

Schedule	Detection levels	1280 px		1536 px	
		mAP@50	mAP@50:95	mAP@50	mAP@50:95
Progressive	P3–P5 (YOLOv11s)	53.25	31.62	54.50	32.55
	P2–P5 (Baseline-P2)	53.32	31.47	54.07	32.01
Single phase	P3–P5 (YOLOv11s)	56.53	34.20	58.47	35.52
	P2–P5 (Baseline-P2)	57.26	34.66	58.84	35.82
Stride-4 level, progressive		+0.07	−0.15	−0.43	−0.54
Stride-4 level, single phase		+0.73	+0.46	+0.37	+0.30
Single phase, P3–P5		+3.28	+2.58	+3.97	+2.97
Single phase, P2–P5		+3.94	+3.19	+4.77	+3.81

sive schedule it was -0.43 points ($[-1.15, 0.26]$, $p = 0.27$). The gain from the stride-4 level was 0.8 points larger under single-phase training than under the progressive schedule ($[-0.08, 1.74]$, $p = 0.07$ at 1536 px). With one seed per configuration and seed-to-seed differences of up to 0.7 points for a single model, we do not treat this interaction as established. The three-level runs also used larger physical batches than the four-level runs under both schedules (Section 4.3).

At the class level, the stride-4 level helped the classes with the smallest boxes. With single-phase training at 1280 px, AP@50 rose by 3.9 points for pedestrians ($[3.1, 4.7]$), 4.2 points for people ($[3.1, 5.4]$), 2.2 points for motorcycles ($[1.3, 3.1]$) and 0.9 points for cars ($[0.6, 1.4]$), all $p < 0.002$. Tricycles and awning-tricycles lost 2.1 and 1.9 points ($p = 0.016$ and 0.028), which does not survive the correction for ten classes, and the remaining classes did not change significantly. Section 5.5 shows that these class differences follow object size.

The stride-4 level is expensive. It adds 0.49M parameters and raises GFLOPs by 74% at every input size (Table 2), because it operates on a feature map with four times as many positions as the stride-8 level. A larger input is the cheaper way to reach the same accuracy: YOLOv11s evaluated at 1536 px scored 58.47% mAP@50 with 124 GFLOPs, more than Baseline-P2 at 1280 px with 150 GFLOPs (57.26%).

Table 6: Adding MEIS and FRM to Baseline-P2 under the full progressive schedule. The paired tests are computed at 1536 px over the 548 validation images.

Seed	Model	1280 px		1536 px		Paired test, mAP@50	
		mAP@50	mAP@50:95	mAP@50	mAP@50:95	95% CI	p (bootstrap / perm.)
0	Baseline-P2	53.32	31.47	54.07	32.01		
	+ MEIS + FRM	53.88	31.96	54.65	32.18		
	Δ	+0.56	+0.49	+0.58	+0.17	$[-0.15, +1.27]$	0.11 / 0.12
42	Baseline-P2	53.60	31.51	54.26	31.91		
	+ MEIS + FRM	53.24	31.21	53.92	31.78		
	Δ	-0.36	-0.30	-0.34	-0.13	$[-1.00, +0.33]$	0.41 / 0.37

The significance tests re-validate both models with batch size one, which changes mAP@50 by at most 0.01 points relative to the table.

5.3. Feature modules

Table 6 adds MEIS and FRM to Baseline-P2 under the full progressive schedule. The change in mAP@50 was +0.56 points with seed 0 and -0.36 points with seed 42 at 1280 px, and +0.58 and -0.34 points at 1536 px. Neither difference is significant (Fig. 6): the paired bootstrap intervals at 1536 px are $[-0.15, 1.27]$ and $[-1.00, 0.33]$, with $p = 0.11$ and $p = 0.41$, and the permutation tests agree ($p = 0.12$ and $p = 0.37$). The mAP@50:95 differences, +0.17 and -0.13 points, are smaller still ($p = 0.48$ and $p = 0.65$). The same test found the schedule effect of Section 5.1 with $p < 0.002$, so the null result does not come from an insensitive test. Even on the favourable seed, the interval excludes improvements larger than about 1.3 points. For this, the modules cost 1.48M parameters and 18% more GFLOPs (Table 2). Broken down by object size (Section 5.5), the modules improved no size range below 32 input pixels on either seed. The seed-0 gain came from objects of at least 32 pixels (+1.7 points, $[0.5, 2.7]$) and did not recur with seed 42 (+0.1 points).

5.3.1. Factorial ablation of MEIS

Table 7 and Fig. 7 separate the two ingredients of MEIS under the shortened schedule. All four MEIS variants include FRM, and the reference model has neither module. The model without modules was the most accurate, and removing either ingredient of MEIS made results slightly better rather than worse. Averaged over paired same-seed differences at 1280 px, multi-scale dilation changed mAP@50 by -0.20 points and input-adaptive gating by -0.24 points, each negative in all four pairings, with a negligible interaction (-0.06). Full MEIS scored 0.44 points below the plain convolution of

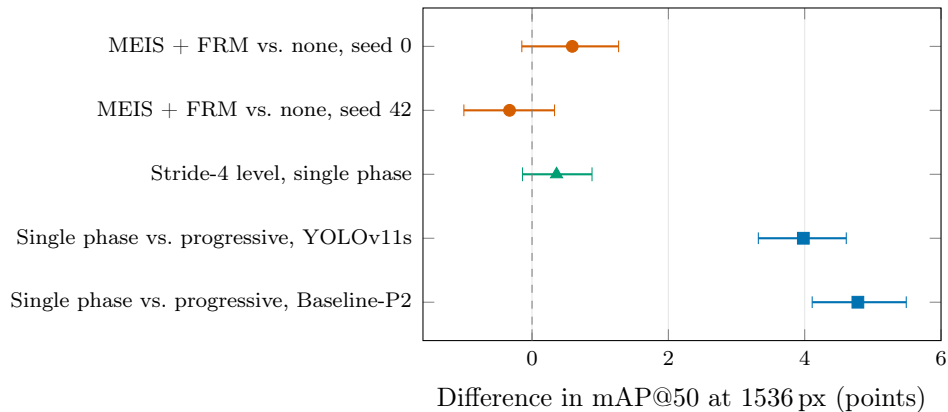


Figure 6: Paired differences with 95% bootstrap confidence intervals over the 548 validation images (seed 0 unless stated). The module and stride-4 intervals include zero; the schedule intervals do not. Point estimates come from the unrounded per-image scores and so can differ by 0.01 points from the difference of the rounded values in Tables 5 and 6.

equal size. At 1536 px the effects pointed the same way (-0.16 and -0.18 points), in three of four pairings each. With two seeds, differences of 0.2 points cannot be resolved statistically; what the ablation does show is that neither ingredient helped.

The module variants trailed the no-module model by 0.6 to 1.0 points in this 75-epoch schedule, whereas under the full schedule the gap was not significant (Table 6). A plausible reading is that the added blocks slow optimization early on and, given enough epochs, settle close to an identity mapping through their residual connections, but we have not tested this directly.

5.4. Seed variation and image sampling

The two sources of uncertainty were of different size. For the MEIS and FRM comparison, the paired difference moved from $+0.58$ to -0.34 points at 1536 px between seeds, a standard deviation of 0.65 points, while the image-level bootstrap standard error was 0.35 points. Individual models also moved: Baseline-P2 scored 53.32% and 53.60% at 1280 px with the two seeds, and the MEIS model 53.88% and 53.24%. The single-phase Baseline-P2 pair moved by a comparable amount, 57.26% and 57.76%, which is why the schedule effect survives the seed change and the module effect does not. Two seeds give only a rough estimate of the seed variance, but the comparison is enough to show that a single seed would have been misleading here. Seed 0 alone

Table 7: Factorial ablation of MEIS on Baseline-P2 with the 75-epoch progressive schedule (mean $\pm$ s.d. over seeds 0 and 42).

Dilations	Gate	Params (M)	1280 px		1536 px	
			mAP@50	mAP@50:95	mAP@50	mAP@50:95
No module		9.92	51.30 $\pm$ 0.10	29.71	51.92 $\pm$ 0.18	30.03
(1,1,1)	static	11.25	50.71 $\pm$ 0.05	29.37	51.18 $\pm$ 0.40	29.79
(1,2,4)	static	11.25	50.54 $\pm$ 0.16	29.15	51.14 $\pm$ 0.16	29.52
(1,1,1)	adaptive	11.40	50.50 $\pm$ 0.11	29.29	51.12 $\pm$ 0.62	29.57
(1,2,4)	adaptive (MEIS)	11.40	50.27 $\pm$ 0.13	29.08	50.84 $\pm$ 0.47	29.26
Dilation (1,2,4) vs. (1,1,1)			-0.20		-0.16	
Adaptive vs. static gate			-0.24		-0.18	
MEIS vs. (1,1,1) with static gate			-0.44		-0.34	

All MEIS variants also contain FRM. Effects in the lower block are means of paired same-seed differences in mAP@50.

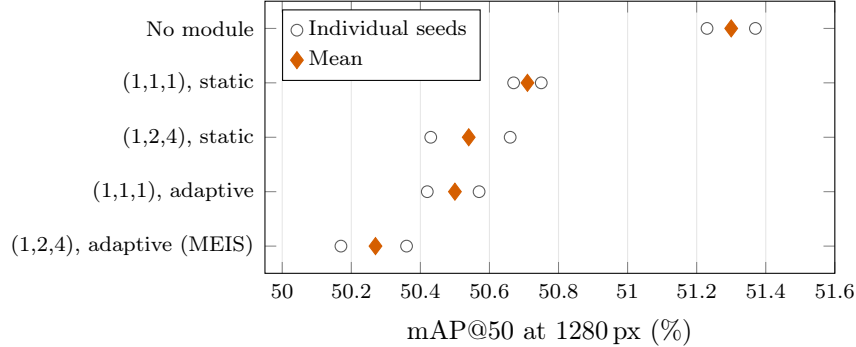


Figure 7: Factorial ablation of MEIS (Table 7). Labels give the branch dilations and the gate type.

suggests that the modules add about half a point; seed 42 suggests that they remove a third of one. For effects below one point we would recommend at least three seeds together with paired tests over the evaluation images.

5.5. Accuracy by object size

Table 8 gives AP@50 in the COCO size ranges, and Fig. 8 shows the three comparisons by object size in network-input pixels, the scale on which the predictions of Section 3 are made.

Detection stride (P1). The stride-4 level changed accuracy where the sampling argument says it should (Fig. 8a). For objects of 4 to 8 input pixels it added 4.0 points AP@50 under the progressive schedule ([1.6, 6.6]) and 6.7

points under single-phase training ($[4.4, 9.0]$), and for objects of 8 to 16 pixels 2.8 and 4.9 points ($[1.6, 3.9]$ and $[3.7, 6.1]$). For objects of 16 to 32 pixels it changed nothing (0.4 and 0.1 points), and for objects of at least 32 pixels it lowered AP@50 by 2.0 and 1.1 points ($[-3.0, -1.0]$ and $[-1.9, -0.3]$). The gain thus disappears once objects span about two stride-8 cells and turns into a loss beyond four, and the losses on larger objects explain why mAP@50 barely moved. Cars, which have at least 655 boxes in every range, show the same pattern on their own: 2.9 to 4.0 points below 16 pixels and less than one point above.

Feature modules (P2). MEIS with FRM changed AP@50 by between -1.2 and $+0.3$ points in the three ranges below 32 pixels on the two seeds (Fig. 8c), and only one of these six intervals excluded zero, the -1.2 points for 8 to 16 pixels with seed 42. Averaged over both seeds, COCO small objects lost 0.8 points ($[-1.5, -0.1]$). The only significant gains, 1.7 points for objects of at least 32 pixels and 1.4 points for COCO medium objects with seed 0, did not recur with seed 42 (0.1 and -0.1 points). Under the same schedule and on the same objects, the stride-4 level gained 2.8 to 4.0 points below 16 pixels. This is what P2 predicts: reweighting features after downsampling did not help the objects that a finer stride did help.

Training schedule (P3). Single-phase training did not concentrate its gain on small objects (Fig. 8b). For YOLOv11s it changed AP@50 by -0.6 and $+0.5$ points for objects below 16 input pixels, neither significant, but added 2.7 points for 16 to 32 pixels and 5.5 points for objects of at least 32 pixels ($[4.6, 6.7]$). In the COCO ranges the gains were 1.6 points for small, 5.2 for medium and 6.1 for large objects. With the stride-4 level, single-phase training also helped objects of 8 to 16 pixels, by 2.7 points, and this exceeded the corresponding gain for YOLOv11s by 2.1 points ($[0.8, 3.5]$). The largest gain was again for objects of at least 32 pixels (6.5 points, $[5.6, 7.6]$). P3 predicted the opposite ordering.

A mismatch between training and test scale could produce a deficit on larger objects: the progressive models saw objects at smaller pixel sizes for 300 of their 350 epochs. If that were the cause, evaluating them at 1024px would raise their AP on medium and large objects. It did not. Medium objects changed by -0.9 points ($[-1.8, -0.1]$) for YOLOv11s and $+0.1$ points ($[-0.8, 0.9]$) for Baseline-P2, and large objects by less than 0.5 points. At 1024px, single-phase training still led on medium objects by 4.5 and 4.1

Table 8: AP@50 at 1280 px by object size, using the COCO area ranges in original-image pixels, with the number of ground-truth boxes in brackets. Size columns average over the classes with at least 100 boxes in the range (all ten for small and medium objects, cars and vans for large objects); the first column is the standard mAP@50.

Schedule	Model	Seed	All sizes	Small [26,586]	Medium [11,105]	Large [1,068]
Progressive	YOLOv11s (P3–P5)	0	53.2	46.1	64.2	78.2
	Baseline-P2	0	53.3	47.0	62.5	79.6
	Baseline-P2	42	53.6	46.6	63.6	77.9
	Baseline-P2 + MEIS + FRM	0	53.9	46.5	63.9	79.4
	Baseline-P2 + MEIS + FRM	42	53.2	45.5	63.5	77.8
Single phase	YOLOv11s (P3–P5)	0	56.5	47.6	69.3	84.2
	Baseline-P2	0	57.3	49.0	68.5	83.2

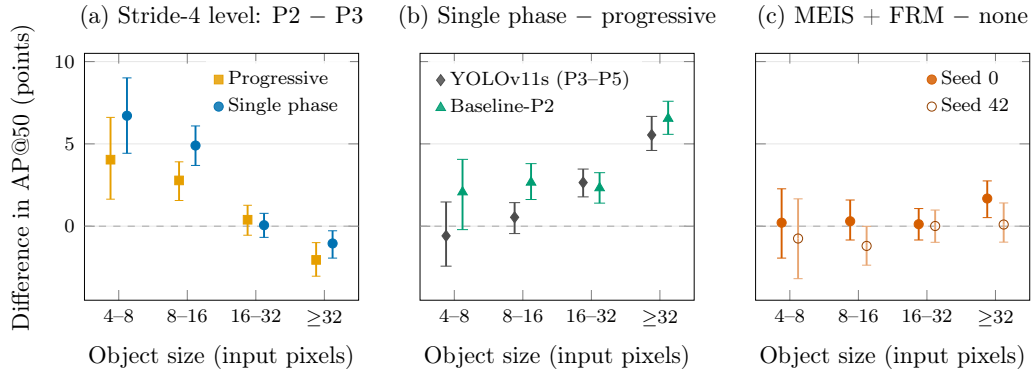


Figure 8: Differences in AP@50 at 1280 px by object size in network-input pixels, with 95% paired bootstrap confidence intervals over the validation images. One stride-4 cell spans 4 pixels and one stride-8 cell 8 pixels. Panels (a) and (b) use seed 0; panel (c) compares models trained with the progressive schedule. Each point averages over the classes with at least 100 boxes in its range.

points (both $p < 0.002$). The single-phase models were, however, more tied to their training size: evaluated at 1024 instead of 1280 px, they lost 1.0 points more than the progressive models on small objects ($[0.3, 1.7]$) and 1.3 points more on medium objects ($[0.7, 2.1]$).

5.6. Cross-dataset evaluation on UAVDT

Table 9 applies the VisDrone-trained models to UAVDT without further training. On cars, the only class annotated in every sequence, the ordering of Sections 5.1 and 5.2 reappears. Single-phase training added 2.24 points AP@50 without the stride-4 level and 2.47 points with it, the stride-4 level added 1.82 points under the progressive schedule and 2.05 points under

Table 9: Zero-shot AP@50 on UAVDT at 1280 px (seed 0), with the number of ground-truth boxes in brackets. No model was trained on UAVDT. The mean weights the three classes equally, so the rare classes dominate it.

Schedule	Model	Car [11,681]	Truck [583]	Bus [186]	Mean AP@50	mAP@50:95
Progressive	YOLOv11s (P3-P5)	72.20	15.84	54.35	47.46	28.87
	Baseline-P2	74.02	24.50	54.20	50.90	30.38
	Baseline-P2 + MEIS + FRM	73.21	13.06	43.33	43.20	25.52
Single phase	YOLOv11s (P3-P5)	74.44	23.06	48.68	48.73	29.01
	Baseline-P2	76.49	20.77	45.22	47.49	28.23

single-phase training, and MEIS with FRM subtracted 0.81 points. None of these differences survives resampling of the eleven sequences: the intervals are $[-0.2, 5.5]$, $[-0.1, 5.7]$, $[-0.5, 4.2]$, $[-0.1, 4.3]$ and $[-2.9, 1.7]$ points. Resampling frames instead would have given intervals several times tighter, and a false impression of precision, because consecutive frames of a video are nearly the same image.

Trucks and buses cannot carry a conclusion here. Trucks appear in six of the eleven sequences with 62% of them in one, and buses in four sequences with 62% in one, so their AP moves by tens of points between resamples and the two classes rank the models in opposite orders. That is also why the three-class mean, which weights them equally with cars, reverses the car ordering.

This evaluation therefore supports a narrow claim: the accuracy ordering on the dominant class survives a change of dataset, and the modules do not help on a second benchmark either. It says nothing about small objects outside VisDrone, since UAVDT annotates only vehicles and its objects are larger (Section 4).

5.7. Relation to published results

Published VisDrone results are difficult to compare with each other because evaluation split, input size, pretraining and inference procedure differ between papers. Table 10 shows the problem with the closest related method. MEIS-YOLO [4] reports 36.0% AP@50 on the VisDrone-DET2019 test set, for a network trained from scratch at 640 px, against 34.6% for its YOLOv11m baseline. Our models were evaluated on the validation set, start from COCO weights and use 1280 px inputs, so the numbers are not on a common scale and the table should not be read as a ranking. The evaluation code matters as well: Ultralytics releases that extend the precision-recall

Table 10: Reported VisDrone-DET2019 results with their evaluation protocols. The rows are not directly comparable.

Method	Split	Input (px)	COCO initialization	AP@50	Params (M)
YOLOv11m, as reported in [4]	test	640	no	34.6	20.04
MEIS-YOLO [4]	test	640	no	36.0	15.04
Baseline-P2, progressive (ours)	val	1280	yes	53.3	9.92
YOLOv11s (P3-P5), single phase (ours)	val	1280	yes	56.5	9.43
Baseline-P2, single phase (ours)	val	1280	yes	57.3	9.92

curve to a recall of one give mAP@50 values 0.5 to 0.7 points higher than the definition we use for the same detections (Section 4.4). Region-based detectors that process magnified crops [16, 17, 18] use several network passes per image and belong to a different accuracy–cost regime.

6. Discussion

6.1. What the sampling account explains

The size breakdown supports two of the three predictions of Section 3. The stride-4 level raised accuracy for the objects the argument singles out, those spanning fewer than two stride-8 cells, and did not help larger ones. MEIS and FRM, which act on features that have already been sampled at stride r , did not improve those same objects on either seed, although a finer stride raised their AP by up to 7 points. Both modules are residual, so they can stay close to an identity mapping when reweighting does not help, and the factorial ablation found no benefit from their multi-scale or adaptive components. Two caveats apply to the module results. Our MEIS block is not the MEIS module of MEIS-YOLO, and the module experiments used the progressive schedule, so whether the modules behave differently when trained at 1280 px throughout remains untested.

The stride-4 level also lowered AP for objects of at least 32 pixels. We did not investigate the cause. One possibility is that the extra level changes how the task-aligned assigner distributes positive samples across levels, which could leave the coarser levels with fewer positives for large objects.

6.2. Why did the progressive schedule do worse?

The schedule effect is the result the sampling account does not explain. If the low-resolution phases had mainly deprived small objects of training targets, the deficit would be largest for small objects. It was largest for

medium and large objects instead, and for YOLOv11s it was absent below 16 input pixels. For Baseline-P2 this is less surprising in hindsight, because even at 640 px only 5.8% of the validation objects are smaller than one stride-4 cell. For YOLOv11s it is harder to reconcile with the account, since 31.0% of the objects fall below one stride-8 cell at 640 px. A mismatch between training and test scale does not explain the deficit either, since the progressive models did not improve on larger objects when evaluated at 1024 px.

The remaining differences between the schedules concern optimization and regularization. The progressive schedule restarts the learning rate with a warm-up at every phase boundary, spends its only 50 epochs at 1280 px with an initial learning rate of 5×10^{-4} , and changes augmentation and loss weights between phases. We did not separate these factors; training a single-resolution schedule with phase-wise restarts, and a progressive schedule without them, would. The one part of the schedule effect that fits the sampling account is that Baseline-P2 gained more than YOLOv11s from single-phase training on objects of 8 to 16 pixels, as expected if the stride-4 level needs high-resolution training to receive targets for small objects. The checkpoint results also rule out faster convergence as an argument for the progressive schedule, since single-phase training matched its final accuracy after 61 epochs.

6.3. Practical recommendations

Five recommendations follow from these experiments. First, train at the input size that will be used at deployment: the single-phase models were the most accurate at their training size but lost more than the progressive models when evaluated at a smaller one. Second, before adding modules, compare the object size distribution in network-input pixels with the finest detection stride. A stride-4 level pays off when many objects span fewer than two stride-8 cells, but it costs accuracy on large objects, 74% more computation and 89% more time, and a larger input size may reach the same accuracy at lower cost. Third, report accuracy by object size, because a small change in mAP can hide opposite effects on small and large objects. Fourth, compare designs with at least two or three seeds, test paired differences over the evaluation images, and report training compute alongside accuracy. Fifth, measure speed where the GPU is the limit, and report the operation count separately rather than in its place. With one image per batch, two of our three models took the same time at 1280 and at 1536 px, because the host could not submit kernels faster than that; read at face value, those numbers

put the cost of the stride-4 level at 18% when it is 89%. A larger batch, or any check that the time grows with the pixel count, separates the model from the machine that runs it.

6.4. Limitations

The study has several limitations. The module comparison was repeated with two seeds, and so were both Baseline-P2 schedules, but the two YOLOv11s cells of the schedule-by-stride comparison rest on one seed each, so the interaction between schedule and stride is not established. The three- and four-level detectors were trained with different physical batch sizes, which changes batch-normalization statistics. Checkpoints were selected on the validation images used for reporting. This makes absolute accuracy slightly optimistic for every model, and selection over up to 350 single-phase epochs may favour those runs somewhat more than selection within the 50-epoch final progressive phase. Size-resolved AP averages only classes with at least 100 boxes in a range, which leaves two classes for large objects. The cross-dataset evaluation covers the 11 UAVDT detection sequences available to us, its truck and bus results rest largely on one sequence each, and UAVDT annotates only vehicles, which are larger than VisDrone’s objects; whether any of this transfers to small non-vehicle objects outside VisDrone is untested. We studied one detector family at one size, so larger models and transformer-based detectors may behave differently; we chose YOLOv11s because it is widely used, has a maintained implementation with pretrained weights, and is a realistic size for drone platforms. The schedule comparison changes learning-rate restarts, augmentation and loss weights together with resolution. The module experiments used the progressive schedule, and the factorial ablation a shortened one. We did not evaluate robustness to motion blur, noise or illumination changes. Finally, the sampling argument is a heuristic, and FPS figures depend on the hardware.

7. Conclusion

For YOLOv11s on VisDrone-DET2019, controlled comparisons separated three kinds of design decision. The finest detection stride changed small-object accuracy in the way a sampling argument predicts: a stride-4 level added 3 to 7 points AP@50 for objects spanning fewer than two stride-8 cells and cost 1 to 2 points on objects larger than 32 pixels, which left mAP almost

unchanged. Edge selection with adaptive gating and CBAM-style recalibration, which act after downsampling, improved no size range below 32 pixels on either of two seeds, and neither multi-scale dilation nor adaptive gating did better than a plain convolution of equal size, while cutting throughput almost in half, from 97 to 53 images per second at 1280 px. Single-resolution training at 1280 px outperformed a four-phase progressive schedule by 3.3 to 4.2 points mAP@50 at the same epoch budget, with and without the stride-4 level and on both seeds of the four-level detector, and matched its accuracy with a third of the compute. The two schedules differ in learning-rate restarts, loss weights and augmentation as well as in resolution, so this implicates the recipe as a whole rather than resolution scheduling on its own; Section 6.2 names the two runs that would separate them. Its gain came mainly from medium and large objects and did not vanish at a smaller test size, so it is better explained by how the two schedules train the network than by how densely they sample small objects. Applied to UAVDT without retraining, the ranking held for cars, the only class annotated throughout, although eleven sequences could not resolve the differences statistically. These results do not show that feature modules or resolution schedules cannot help small-object detection. They show that such changes should be compared with baselines trained at the same resolution and budget, across seeds, with paired tests and a breakdown by object size, before a gain is attributed to the mechanism it was designed for.

Acknowledgements

The authors thank the VisDrone and UAVDT teams for the benchmarks and the Ultralytics team for the YOLOv11 implementation.

Appendix A. Earlier single-seed ablations

Before the controlled experiments above, we ran a set of single-seed ablations with an earlier configuration. It differed from the main experiments in four ways: the detectors used three detection levels (P3 to P5), module variants did not receive copied head weights and so started with randomly initialized necks while the reference neck was pretrained, models were evaluated at 1024 px, and at most 300 detections were kept per image. Table A.1 reports these runs for completeness, including the placement of FRM before

Table A.1: Earlier ablations with three detection levels, the full progressive schedule and evaluation at 1024px. One seed per row unless stated.

Configuration	mAP@50	mAP@50:95
PANet neck, no module	49.84	29.61
PANet neck + MEIS	48.15	27.96
Weighted fusion (three levels at P4) + MEIS	47.91	27.96
+ FRM after fusion, $\alpha = 0.5$ (three runs)	48.50 ± 0.28	28.47 ± 0.27
+ FRM before fusion, $\alpha = 0.5$	48.02	28.15
+ FRM after fusion, $\alpha = 0.3$	47.90	28.13
+ FRM after fusion, $\alpha = 0.7$	48.91	28.62
Weighted fusion (two levels at P4) + MEIS + FRM	49.85	29.56

or after fusion, the residual blend factor α , and two- versus three-level fusion at P4.

No module configuration exceeded the plain PANet neck. Three repeated runs of the same configuration spanned 48.25% to 48.80% mAP@50, and most other differences among module variants lie within about twice that spread. Two-level fusion scored 1.35 points above the mean of those three runs of the matching three-level configuration, the opposite of what simultaneous three-level fusion was designed to achieve, and equalled the no-module neck. Because the module variants in this configuration started from randomly initialized necks and each was run once, we do not draw conclusions from these numbers beyond the absence of a gain over the PANet neck.

References

- [1] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, H. Ling, Detection and tracking meet drones challenge, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (11) (2021) 7380–7399.
- [2] M. A. Khan, W. Ectors, T. Bellemans, D. Janssens, G. Wets, Object detection and tracking methods for uav-based aerial surveillance: A survey, *IEEE Access* 7 (2019) 53388–53409.
- [3] X. Zhu, S. Lyu, X. Wang, Q. Zhao, TPH-YOLOv5: Improved YOLOv5 based on transformer prediction head for object detection on drone-captured scenarios, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, 2021, pp. 2778–2788.

- [4] Y. Liu, J. Wu, L. Chen, MEIS-YOLO: Improving YOLOv11 for efficient aerial object detection with lightweight design, *Intelligent and Converged Networks* 6 (2) (2025) 151–163. doi:10.23919/ICN.2025.0010.
- [5] Q. Ding, P. Li, X. Yan, D. Shi, L. Liang, W. Wang, H. Xie, J. Li, M. Wei, CF-YOLO: Cross fusion YOLO for object detection in adverse weather with a high-quality real snow dataset, *arXiv preprint arXiv:2206.01381* (2022).
- [6] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, P. Vincent, Accounting for variance in machine learning benchmarks, in: *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 3, 2021, pp. 747–769.
- [7] Ultralytics, Ultralytics yolo11, <https://docs.ultralytics.com/models/yolo11/> (2024).
- [8] M. Tan, R. Pang, Q. V. Le, Efficientdet: Scalable and efficient object detection, in: *CVPR*, 2020.
- [9] S. Woo, J. Park, J.-Y. Lee, I. S. Kweon, Cbam: Convolutional block attention module, in: *Proceedings of the European Conference on Computer Vision*, 2018, pp. 3–19.
- [10] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie, Feature pyramid networks for object detection, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 2117–2125.
- [11] S. Liu, L. Qi, H. Qin, J. Shi, J. Jia, Path aggregation network for instance segmentation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8759–8768.
- [12] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7132–7141.

- [13] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, Q. Hu, Eca-net: Efficient channel attention for deep convolutional neural networks, in: CVPR, 2020.
- [14] H. Zhang, K. Liu, Z. Gan, G.-N. Zhu, UAV-DETR: Efficient end-to-end object detection for unmanned aerial vehicle imagery, arXiv preprint arXiv:2501.01855 (2025).
- [15] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen, DETRs beat YOLOs on real-time object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 16965–16974.
- [16] F. Yang, H. Fan, P. Chu, E. Blasch, H. Ling, Clustered object detection in aerial images, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.
- [17] C. Li, T. Yang, S. Zhu, C. Chen, S. Guan, Density map guided object detection in aerial images, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2020, pp. 190–191.
- [18] Y. Huang, J. Chen, D. Huang, UFPMP-Det: Toward accurate and efficient object detection on drone imagery, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2022.
- [19] C. Yang, Z. Huang, N. Wang, QueryDet: Cascaded sparse query for accelerating high-resolution small object detection, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022.
- [20] F. C. Akyon, S. O. Altinuc, A. Temizel, Slicing aided hyper inference and fine-tuning for small object detection, in: Proceedings of the IEEE International Conference on Image Processing (ICIP), 2022.
- [21] Y. Bengio, J. Louradour, R. Collobert, J. Weston, Curriculum learning, in: Proceedings of the 26th Annual International Conference on Machine Learning (ICML), ACM, 2009, pp. 41–48. [doi:10.1145/1553374.1553380](https://doi.org/10.1145/1553374.1553380).

- [22] M. Tan, Q. V. Le, EfficientNetV2: Smaller models and faster training, in: Proceedings of the 38th International Conference on Machine Learning (ICML), 2021.
- [23] H. Touvron, A. Vedaldi, M. Douze, H. Jégou, Fixing the train-test resolution discrepancy, in: Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [24] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.
- [25] T. Berg-Kirkpatrick, D. Burkett, D. Klein, An empirical investigation of statistical significance in NLP, in: Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), 2012.
- [26] C. E. Shannon, Communication in the presence of noise, Proceedings of the IRE 37 (1) (1949) 10–21.
- [27] R. Zhang, Making convolutional networks shift-invariant again, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019.
- [28] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, W. Huang, TOOD: Task-aligned one-stage object detection, in: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021.
- [29] D. Du, Y. Qi, H. Yu, Y. Yang, K. Duan, G. Li, W. Zhang, Q. Huang, Q. Tian, The unmanned aerial vehicle benchmark: Object detection and tracking, in: European Conference on Computer Vision (ECCV), 2018, pp. 375–391.
- [30] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, Microsoft coco: Common objects in context, in: European Conference on Computer Vision, Springer, 2014, pp. 740–755.